

Homework 03

Kristina

April 6, 2026

4.3

算法设计思想：贪心法，从 A 到 B 相隔尽量远设立基站

Algorithm 1 LocateStation(d, n)

Input: 数组 $d[1..n]$ ($d[1] < d[2] < \dots < d[n]$), 正整数 n

Output: 基站建立的位置数组 x

```

1:  $x[1] \leftarrow d[1] + 4$ 
2:  $k \leftarrow 1$ 
3: for  $i = 1$  to  $n$  do
4:   if  $x[k] + 4 < d[i]$  then
5:      $x[k + 1] \leftarrow d[i] + 4$ 
6:      $k \leftarrow k + 1$ 
7:   end if
8: end for
9: return  $x$ 

```

正确性证明：

对于另一种符合要求的基站设置方式 y ($y[i] < y[j] \ \forall i < j$), 将其与 $x[1..m]$ 进行比较

- 若 $y[i] = x[i] \ \forall i \in \{1, 2, \dots, m\}$, 则 y 中元素个数 $\geq m$
- 若 $\exists i \in \{1, 2, \dots, m\}$ s.t. $y[i] \neq x[i]$, 则取最小的一个 i 进行讨论
 - 若 $y[i] > x[i]$, 设 $d[j] = x[i] - 4$, 由 $x[i] = d[j] + 4$, $x[i-1] + 4 < d[j]$, $x[i-1] = y[i-1]$, 得 $y[i-1] + 4 < d[j] < y[i] - 4$, 即 $d[j]$ 位置的房子附近没有基站, 不符合要求
 - 若 $y[i] < x[i]$, 类似可得 $y[i+1] > x[i+1]$ (若数组 y 没有第 j 项则 $y[j] = x[m] + 1$) 时不符合要求, 对于 $y[i+1] = x[i+1]$ 和 $y[i+1] < x[i+1]$ 的情况, 可分别递推归纳得出 $y[m] \leq x[m]$, 则 y 至少有 m 项

由此可知, 对于任意一种符合要求的基站设置方式, 其所需基站总数不少于贪心法所得出数组 x 中的总数, 故此算法正确

算法时间由循环决定, 循环运行 n 次, 循环体内时间复杂度 $O(1)$, 故算法在最坏情况下的时间复杂度为 $O(n)$

4.6

贪心法，将所有作业按加工时间排序后从小到大完成

正确性证明：

设作业 i 在调度中的次序为 $s(i)$ ，平均完成时间

$$T = \frac{1}{n} \sum_{i=1}^n w(i) = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^{s(i)} t(j) = \frac{1}{n} \sum_{i=1}^n (n - s(i) + 1) t(i) = \frac{n+1}{n} \sum_{i=1}^n t(i) - \frac{1}{n} \sum_{i=1}^n s(i) t(i)$$

$$\min T \iff \max \sum_{i=1}^n s(i) t(i)$$

由排序不等式，当 s 满足 $s(i) > s(j) \forall t(i) < t(j)$ 时， $\sum_{i=1}^n s(i) t(i)$ 取到最大值，即调度 f 按照加工时间从小到大安排作业任务

此时算法需要对所有作业时间进行排序，最坏情况下的时间复杂度为 $O(n \log n)$

4.13

截止时间内至多可以完成 $\lfloor D \rfloor$ 项作业，将罚款最小的 $n - \lfloor D \rfloor$ 项放在最后（无需排序），可以使得总罚款最少

算法步骤：

- 调用 Select 函数找出数组 m 中第 $\lfloor D \rfloor + 1$ 小的元素 M
- 遍历数组 $m[i]$ ，对于其中 $< M$ 的元素，其调度从 1 开始正向排序；对于其中 $\geq M$ 的元素，其调度从 n 开始反向排序

正确性证明：此算法默认作业不间断执行，若相邻两项中间有时间间隔，则第 $\lfloor D \rfloor$ 项完整延后，总罚款不少于原算法；在不间断的情况下，若任意交换原调度中两个作业 i, j ($f(i) < f(j)$) 的次序，当二者同属于前 $\lfloor D \rfloor$ 项或同属于后 $n - \lfloor D \rfloor$ 项时不影响罚款情况，当分别属于两个集合时由 $m(i) < M \leq m(j)$ 得总罚款增量 $m(j) - m(i) > 0$ ，总罚款增加，故算法能够求出使得总罚款最少的调度

算法中的两步均可以在 $O(n)$ 时间内完成，故算法在最坏情况下的时间复杂度为 $O(n)$

4.18

(1)

算法步骤：

- 初始情况下 $A = \{1\}$ ，用 j 表示 A 中当前标号最大的广告
- 从 2 到 n 遍历广告集合，若第 i 条广告满足 $s(i) \geq d(j)$ ，则将 i 加入集合 A 中并更新 j

正确性证明：

- 对于 $1 \notin A$ 的最优解, 设 A 中最小元素为 i , 则由于 $d(1) \leq d(i)$, 用 $A' = (A - \{i\}) \cup \{1\}$ 替代 A 不改变总效益, 仍为最优解之一, 故只需考虑包含 1 的最优解集合
- 对于任意已确定的 A 的部分子集 B , 设 B 中最大元素为 j , S 中使得 $s(i) \geq d(j)$ 的元素构成集合 T , 则问题等价于在 T 中选择一组不互斥的广告使其总收益最大, 类似上一步可得任意最优解可转化为包含 T 中最小元素的最优解, 即只需考虑包含 T 中最小元素的最优解集合
- 归纳可得, 该算法能够求出使得任意两个广告都相容且总收益最大的集合 A

时间复杂度由循环步骤决定, 循环运行 n 次, 循环体内时间复杂度 $O(1)$, 故算法时间复杂度为 $O(n)$

(2)

使用动态规划算法, 逐项计算使用前 k 条广告时所能达到的最大收益

$$dp[k] = \max\{dp[k-1], dp[t(k)] + v(k), dp[1] = v(1)\}$$

其中 $t(k)$ 表示使得 $d(j) \leq s(k)$ 的最大的 j

算法步骤:

- 对于每一个 i , 逐项查找出 $t(i)$ 的值
- 从 1 到 n 计算出使用当前前几条广告的最大收益, 并记录此时的最优解集合

算法时间复杂度由计算 $t(i)$ 和计算 $dp[k]$ 两步构成, 第一步复杂度 $O(n^2)$, 第二步复杂度 $O(n)$, 故最坏情况下的时间复杂度为 $O(n^2)$

5.2

设 $\langle x_1, x_2, x_3, x_4 \rangle$ 表示 4 种配件的供应商标号, $x_i \in \{1, 2, 3\}$

$$\min \sum_{i=1}^4 w_{ix_i}$$

$$\sum_{i=1}^4 v_{ix_i} \leq 120$$

搜索空间是 3 叉树, 深度为 4, 第 i 层代表对第 i 个零件的供应商做出选择

深度优先遍历, 并同时记录当前的总价格 cp 、总重量 cw 和最小总重量 $bestw$ (初始设为 $+\infty$)

搜索时对树进行剪枝: 如果当前价格 cp + 下一个零件的最小价格 > 120 , 则停止向下搜索; 如果当前重量 cw 已经 $\geq bestw$, 则即便后面选最轻的也不可能优于当前最优解, 直接剪枝

解得 $\langle x_1, x_2, x_3, x_4 \rangle = \langle 3, 1, 2, 3 \rangle$, 此时总重量 31, 总价值 119

5.5

可行解 $\langle x_1, x_2, \dots, x_8 \rangle$ 表示第 i 行的棋子所在列，搜索空间是 8 叉树

对其进行广度优先搜索，在每一层对与先前摆放的棋子同行、同列、同斜线的情况剪枝

对于 n 皇后问题，算法最坏情况下的时间复杂度为 $O(n^n)$

5.6

用 $\langle x_1, x_2, \dots, x_n \rangle$ 表示所求子集 A ，其中 $x_i \in \{0, 1\}$ ， $x_i = 1$ 表示 $a_i \in A$ ， $x_i = 0$ 表示 $a_i \notin A$

先对这 n 个正数从大到小排序 $(b_n, \dots, b_1)$ ，并从其中最大的 $\leq M$ 的数开始搜索

对于每个数，可以选择在 A 中和不在 A 中，搜索空间为二叉树（子集树）

剪枝的情况：对于已有的和为 m 的集合，若 $b_i + m > M$ ，则停止搜索 $b_i \in A$ 的分支；若

$\sum_{j=i-1}^1 b_j + m < M$ ，则停止搜索 $b_i \notin A$ 的分支

每一层有两个分支，算法最坏情况下的时间复杂度为 $O(2^n)$

5.7

设可行解 $\mathbf{X} = (x_1, x_2, \dots, x_n)$ 为 $\{1, 2, \dots, n\}$ 的一个排列， $x_i = j$ 表示将第 j 件工作分配给第 i 个人

$$\min \sum_{i=1}^n C(i, x_i)$$

$$x_i \in \{1, \dots, n\}, \quad x_i \neq x_j$$

搜索空间是排列树，第 i 层有 $n + 1 - i$ 个分支

通过深度优先遍历排列树，并记录当前已知最小总成本 min_cost ，设 cp 为前 i 个人已分配工作的成本之和，如果当前 $cp \geq min_cost$ ，则停止搜索并回溯

搜索的节点总数为 $O(n!)$ ，每个节点需要 $O(n)$ 时间检查该工作是否已被分配，算法最坏情况下的时间复杂度为 $O(nn!)$